

Notes Week 7

a. Results TPAGB Grid Week 6

Presumably due to the very low `envelope_mass_limit` of 0.0001, many of the TPAGB mass transfer models took very long to run.

Since the evolution from the end of the TPAGB phase to a white dwarf is not important for mass transfer, this extra computation is unnecessary. I ended the pending jobs from last week and let the remaining models finish¹. This resulted in a total of 30 models.

Although 30 is not an unmanageable amount to manually load, I think, also with the future in mind, I could really use a nice `MesaGrid` class to handle the loading and retrieving of mesa models in grids.

¹ Either until the star is a white dwarf, or the 2 days wall time had exceeded.

a.1 MesaGrid Class

The `MesaGrid` class has a few core features.

- Loading all, or specific binary and star `history.data` files.
- Sorting history data.
- Iteratively retrieving all models.
- Iteratively retrieving models with equal q or R_{RL} .
- Retrieving a specific model.
- Accessing reference single star history data
- Aligning binary history data with single star history data.

To achieve this, I use two classes. A `MesaModel` class, used to store the important data and properties for the mesa models and the `MesaGrid` class.

The `MesaModel` class is very simple and is only used to conveniently access model data.

```
class MesaModel:
    def __init__(self, binary_path=None, tpagb_path=None, initial_age=None):

        self.binary = mr.MesaData(str(binary_path)) if binary_path else None
        self.star = mr.MesaData(str(tpagb_path)) if tpagb_path else None
        self.initial_age = initial_age
        self.age = self.star.star_age if tpagb_path else None
        self.env_mass = (
            self.star.star_mass - self.star.he_core_mass if tpagb_path else None
        )

    def __str__(self):
        if self.star:
            return f"MESA model with R = {self.star.R[0]:.2f}, \
                R_RL = {self.star.rl_1[0]:.2f} and \
                q = {(self.star.star_2_mass[0] / self.star.star_1_mass[1]):.3f}"

            return f"Empty MESA model"

    def set_initial_age(self, age):
        self.initial_age = age
        self.age = self.star.star_age + self.initial_age
```

The `MesaGrid` class is much longer, so I will only show the most important parts:

```
class MesaGrid:
    def __init__(self, grid_dir, R1=None, q=None):
        ...
        self.R1_filter = self._normalize_filter(R1)
        self.q_filter = self._normalize_filter(q)

        self._get_tpagb_age()
        self._load_references()

        self._load_models()
        self._finalize_grid()

    def _load_references(self):
        ...

    def _get_tpagb_age(self):
        ...

    def _normalize_filter(self, val):
        ...
        return {filter}

    def _passes_filter(self, R1, q):
        ...
        return bool

    def _parse_name(self, name):
        ...
        return R1, q

    def _load_models(self):
        ...
        for run_dir in run_dirs:
            ...
            parsed = self._parse_name(run_dir.name)
            ...
            R1, q = parsed

            if not self._passes_filter(R1, q):
                continue

            ...
            model = MesaModel(
                binary_path,
                tpagb_path,
            )

            age = self._find_initial_age(model)
            model.set_initial_age(age)

            self.models[(R1, q)] = model
            self.R1_vals.add(R1)
            self.q_vals.add(q)

    def _find_initial_age(self, model):
        ...
        return model_age - self.tpagb_age

    def _finalize_grid(self):
        ...

    def __getitem__(self, key):
        return self.models[key]

    def get(self, R1, q):
        return self.models.get((R1, q))

    def iter_models(self):
        for R1, q in self.sorted_keys:
            yield R1, q, self.models[(R1, q)]

    def get_R1_slice(self, R1):
        return [
            (q, self.models[(R1, q)])
            for q in self.q_vals
            if (R1, q) in self.models
        ]

    def get_q_slice(self, q):
        return [
            (R1, self.models[(R1, q)])
            for R1 in self.R1_vals
            if (R1, q) in self.models
        ]
```

Set the grid directory and filters. Call internal functions to load the models and references.

Load the single star reference histories and save them.

Find the reference star age at the start of the TPAGB.

Normalises input of R1 and q into a set.

Returns whether the requested model passes the user given filter.

Reads R_1 and q from the model run directory and returns these values.

For all models, parses the directory name, checks if the model passes the user defined filter and reads the binary and single star history. It then computes the initial age of the model and saves it to a last of all models. It also adds the parsed info into the sets of q and R_1.

Finds the initial age of the model relative to the TPAGB by comparing the first entry of the history data with the reference single star history.

Sorts the keys so they are ordered first by ascending Roche lobe radius, then by ascending q. Allows easy access of specific model.

Also access specific model, no error if the model does not exist.

Iterate in sorted order: R1 ascending, then q ascending

Return all models with given R1 sorted by q

Return all models with given mass ratio sorted by R1.

a.2 Demo

Below, I show some use cases:

```
grid = MesaGrid("../grid-dir/", q=0.500)
print(grid.models)
```

```
{(225.0, 0.5): <scripts.general_utils.mesa_grid.MesaModel object at 0x7ffc3aa27b0>,
 (150.0, 0.5): <scripts.general_utils.mesa_grid.MesaModel object at 0x7ffd6e10410>,
 (375.0, 0.5): <scripts.general_utils.mesa_grid.MesaModel object at 0x7ffd6e11090>,
 (300.0, 0.5): <scripts.general_utils.mesa_grid.MesaModel object at 0x7ffd7f31350>}
```

We can iterate over these models using the `grid.iter_models()` function:

```
for R1, q, model in grid.iter_models():
    print(f"R1 = {R1}, {q = }, {model}")

R1 = 150.0, q = 0.5, MESA model with R = 116.62, R_RL = 150.00 and q = 0.500
R1 = 225.0, q = 0.5, MESA model with R = 116.62, R_RL = 225.00 and q = 0.500
R1 = 300.0, q = 0.5, MESA model with R = 213.79, R_RL = 300.00 and q = 0.500
R1 = 375.0, q = 0.5, MESA model with R = 272.64, R_RL = 375.00 and q = 0.500
```

Loading a 2d grid is also easy:

```
grid = MesaGrid(
    "../grid-dir/",
    q=[0.125,0.250,0.375],
    R1=[150,225,300])
for R1, q, model in grid.iter_models():
    print(f"R1 = {R1}, {q = :.3f}, {model}")
```

```
R1 = 150.0, q = 0.125, MESA model with R = 116.62, R_RL = 150.00 and q = 0.125
R1 = 150.0, q = 0.250, MESA model with R = 116.62, R_RL = 150.00 and q = 0.250
R1 = 150.0, q = 0.375, MESA model with R = 116.62, R_RL = 150.00 and q = 0.375
R1 = 225.0, q = 0.125, MESA model with R = 116.62, R_RL = 225.00 and q = 0.125
R1 = 225.0, q = 0.250, MESA model with R = 116.62, R_RL = 225.00 and q = 0.250
R1 = 225.0, q = 0.375, MESA model with R = 116.62, R_RL = 225.00 and q = 0.375
R1 = 300.0, q = 0.125, MESA model with R = 213.79, R_RL = 300.00 and q = 0.125
R1 = 300.0, q = 0.250, MESA model with R = 213.79, R_RL = 300.00 and q = 0.250
R1 = 300.0, q = 0.375, MESA model with R = 213.79, R_RL = 300.00 and q = 0.375
```

The grid can be sliced using the `get_q_slice` and `get_R1_slice` functions:

```
print("slicing q")
for R1, model in grid.get_q_slice(q=0.125):
    print(model)

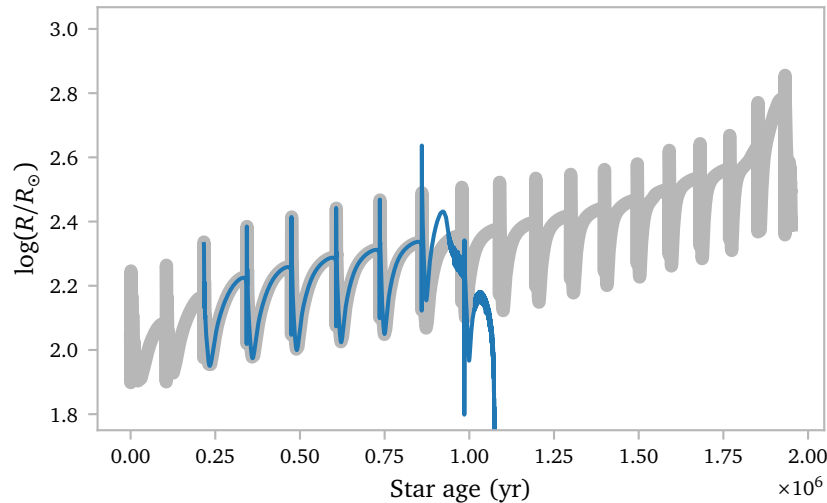
print("\nslicing R1")
for R1, model in grid.get_R1_slice(R1=225):
    print(model)
```

```
slicing q
MESA model with R = 116.62, R_RL = 150.00 and q = 0.125
MESA model with R = 116.62, R_RL = 225.00 and q = 0.125
MESA model with R = 213.79, R_RL = 300.00 and q = 0.125
```

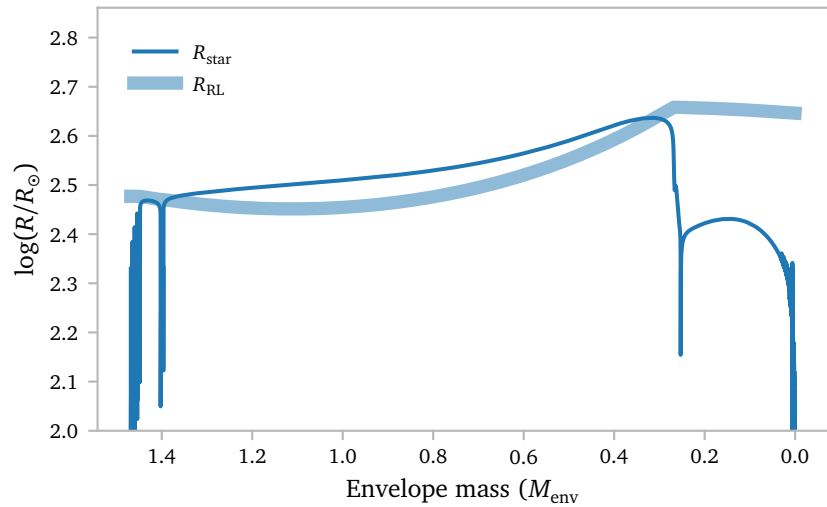
```
slicing R1
MESA model with R = 116.62, R_RL = 225.00 and q = 0.125
MESA model with R = 116.62, R_RL = 225.00 and q = 0.250
MESA model with R = 116.62, R_RL = 225.00 and q = 0.375
```

Using the `MesaGrid` class, we can select the reference single star history. It is also possible to select a single model from the grid:

```
model = grid.get(R1=300, q=0.875)
plt.plot(grid.ref_tpagb.star_age, grid.ref_tpagb.R)
plt.plot(model.age, model.star.R)
plt.show()
```



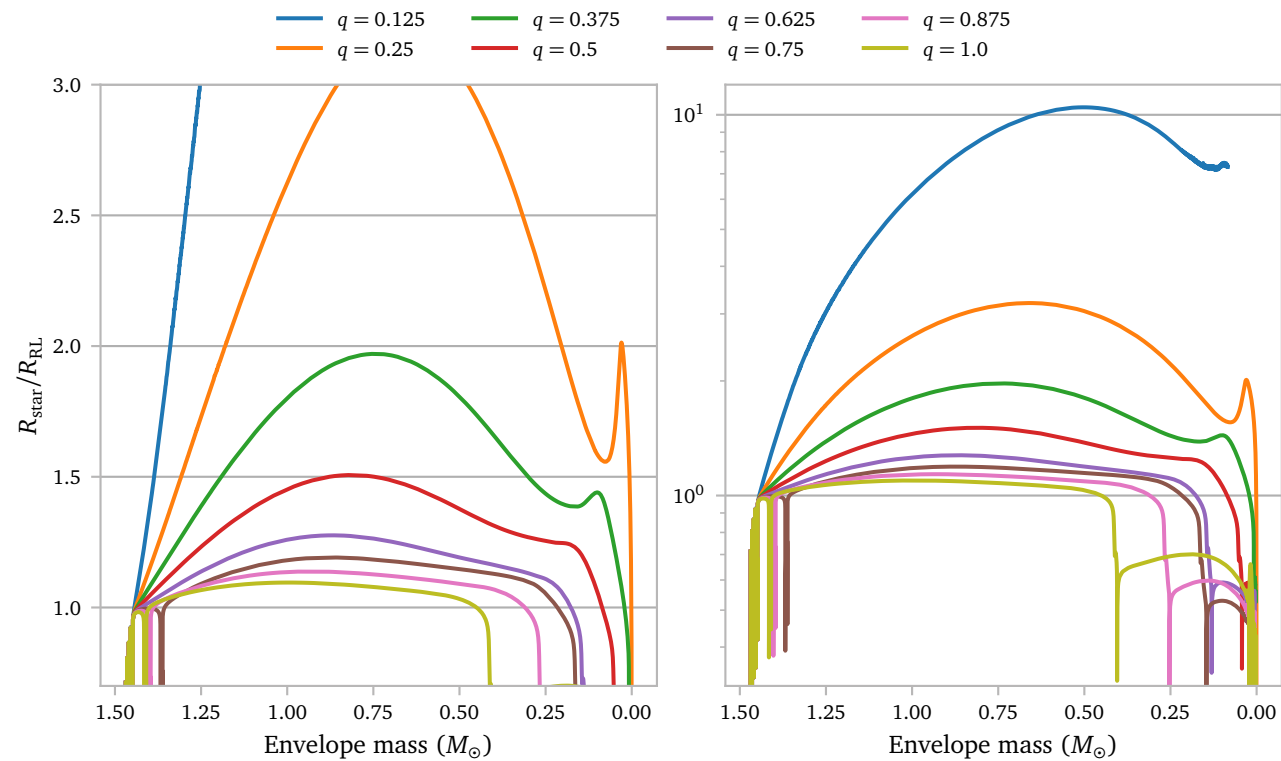
```
model = grid.get(R1=300, q=0.875)
plt.plot(model.env_mass, model.star.R)
plt.plot(model.env_mass, model.star.rl_1)
plt.gca().invert_xaxis()
plt.show()
```



a.3 Results

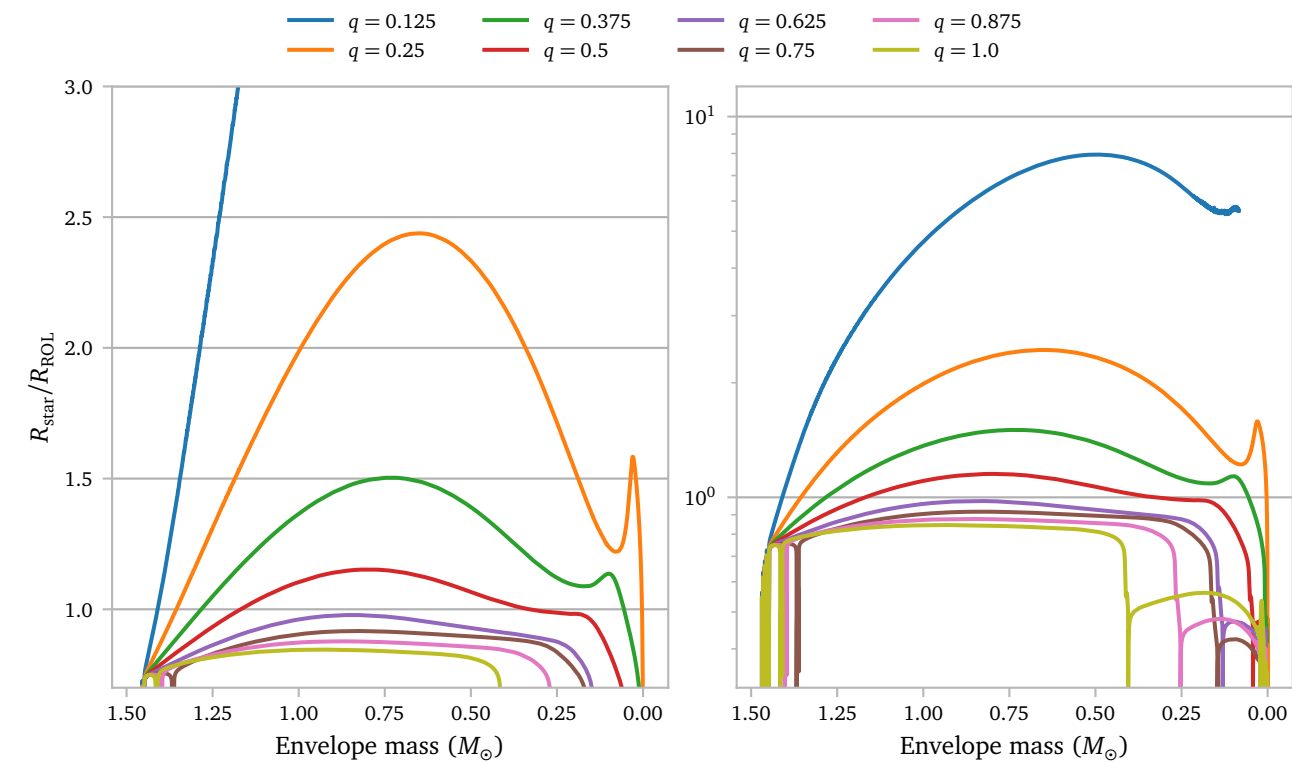
Since I ended last week's run early, I only had a complete set of models for $R_{RL} = 300R_{\odot}$. Therefore, I will plot some (hopefully) interesting things using these models.

Below, I plot the relative Roche lobe sizes as a function of envelope mass for different q and a starting Roche lobe radius of $R_{RL} = 300R_{\odot}$.

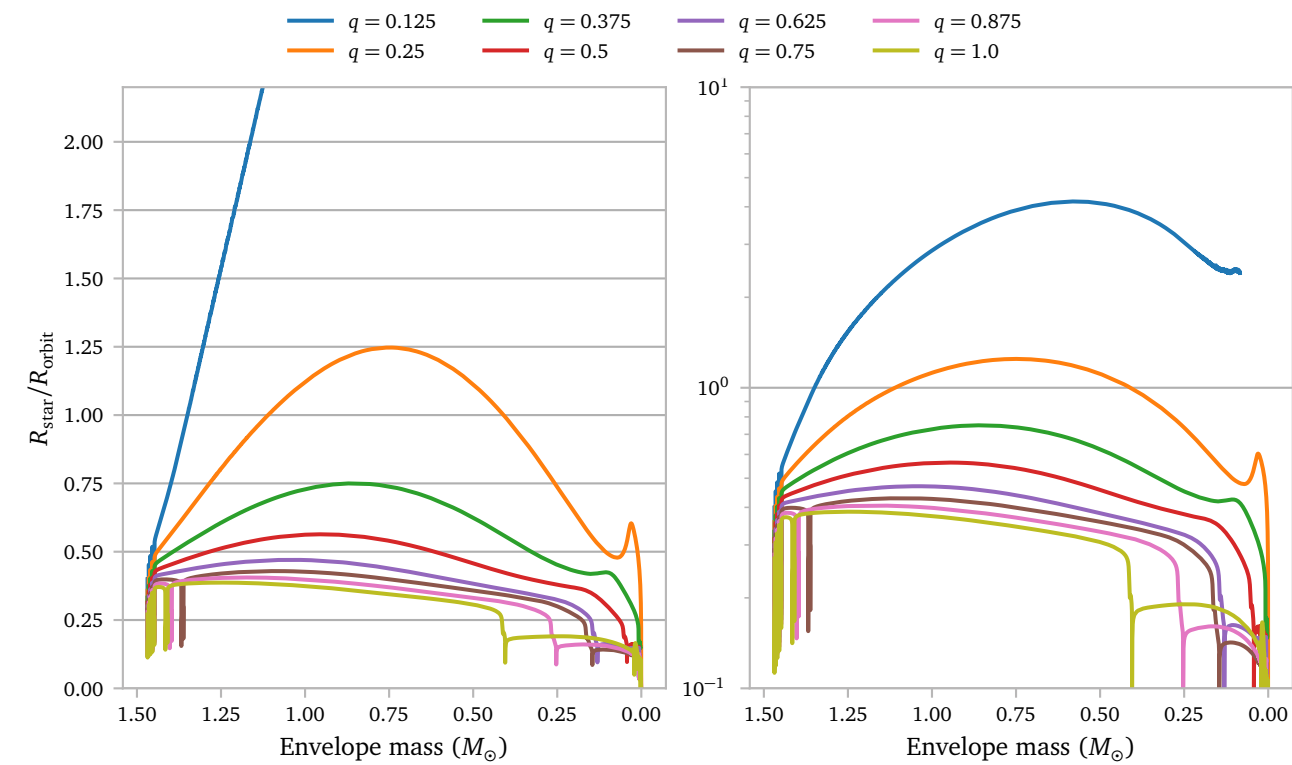


I think this nicely shows how important the mass ratio is for the resulting maximum R_{star}/R_{RL} .

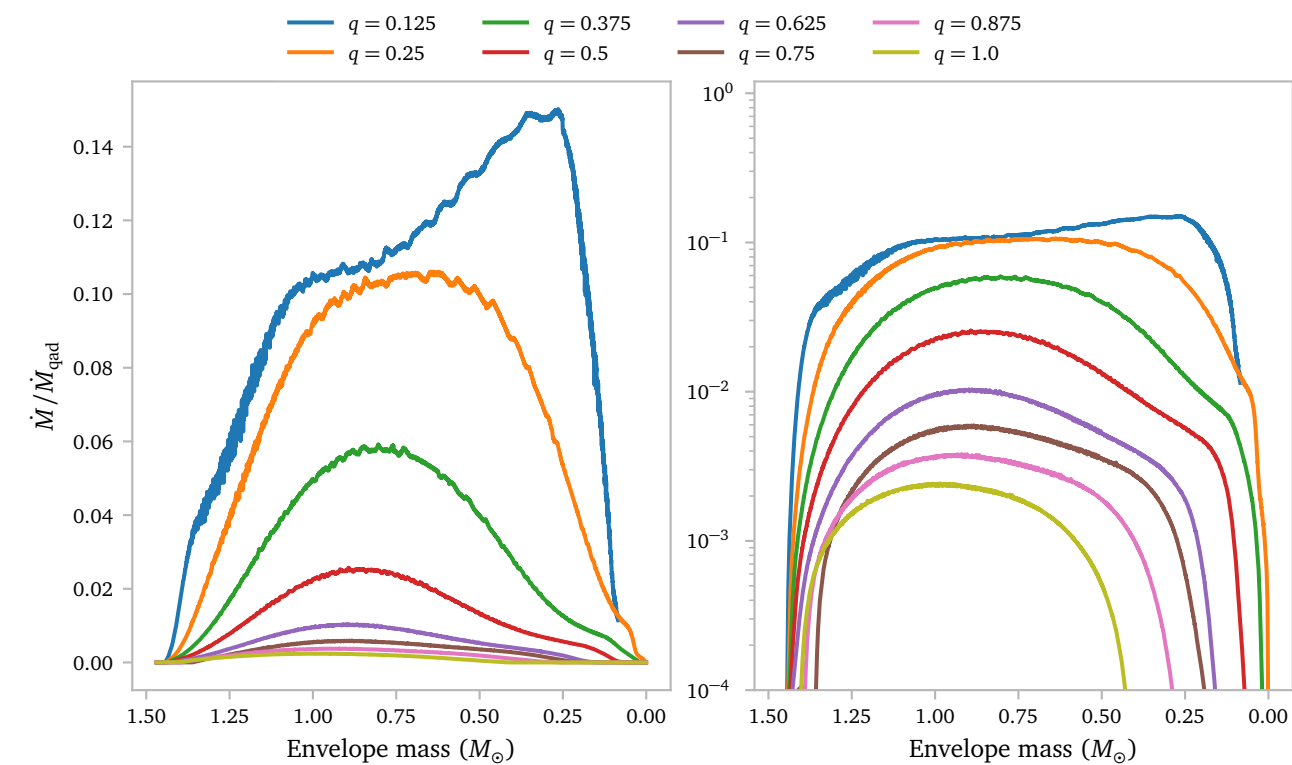
In a similar way, using the analytic approximation for the outer Roche lobe radius from [Temmink et al. \(2023\)](#), the effects of q on the ratio $R_{\text{star}}/R_{\text{ROL}}$ can be plotted. Here, I shade the region above this value, the MESA simulations are not reliable.



Similarly, the radius of the star and the radius of the orbit can be compared. When the ratio exceeds 2, the system must be a contact binary, even for the smallest accretor.



Then, the ratio of actual mass loss over the quasi adiabatic limit. Here a value of 1 should mean mass loss cannot be stable.



Clearly, all models stay well below 1.

b. Overshooting

I compare the single star evolution of stars with different masses for 3 different overshooting prescriptions:

1. Overshooting from [Rees et al. \(2024\)](#)
2. Overshooting from [Temmink et al. \(2023\)](#)
3. Overshooting from [Temmink et al. \(2023\)](#) with different $f_{\text{ov},0}$.

Below, I note the exact inlist settings for all three cases.

b.1 Rees et al. (2024)

```
! inlist_MS
&controls
...
overshoot_scheme(1) = 'step' !jermyn et al 2022
overshoot_zone_type(1) = 'any'
overshoot_zone_loc(1) = 'core'
overshoot_bdy_loc(1) = 'any'
overshoot_f(1) = 0.129
overshoot_f0(1) = 0.0129
overshoot_D0(1) = 0d0
overshoot_Delta0(1) = 1d0
overshoot_scheme(2) = 'exponential'
overshoot_zone_type(2) = 'nonburn'
overshoot_zone_loc(2) = 'shell'
overshoot_bdy_loc(2) = 'any'
overshoot_f(2) = 0.0174
overshoot_f0(2) = 0.00174
overshoot_D0(2) = 0d0
overshoot_Delta0(2) = 1d0
...

! inlist_GB
&controls
...
overshoot_scheme(1) = 'exponential'
overshoot_zone_type(1) = 'nonburn'
overshoot_zone_loc(1) = 'shell'
overshoot_bdy_loc(1) = 'any'
overshoot_f(1) = 0.0174
overshoot_f0(1) = 0.00174
overshoot_D0(1) = 0d0
overshoot_Delta0(1) = 1d0
...

! inlist_CHeB
&controls
...
predictive_mix(1) = .true.
predictive_zone_type(1) = 'any'
predictive_zone_loc(1) = 'core'
predictive_bdy_loc(1) = 'any'
predictive_superad_thresh(1) = 0.005
predictive_avoid_reversal = 'he4'

overshoot_scheme(2) = 'exponential'
overshoot_zone_type(2) = 'nonburn'
overshoot_zone_loc(2) = 'shell'
overshoot_bdy_loc(2) = 'any'
overshoot_f(2) = 0.0174
overshoot_f0(2) = 0.00174
overshoot_D0(2) = 0d0
overshoot_Delta0(2) = 1d0
...

! inlist_EAGB
&controls
...
overshoot_scheme(1) = 'exponential' ! IPCZ
  ↳ overshooting controls
overshoot_zone_type(1) = 'burn_He'
overshoot_zone_loc(1) = 'shell'
overshoot_bdy_loc(1) = 'bottom'
overshoot_f(1) = 0.008 ! Herwig 2008
overshoot_f0(1) = 0.0008
overshoot_scheme(2) = 'exponential' ! envelope
  ↳ overshooting
overshoot_zone_type(2) = 'nonburn'
overshoot_zone_loc(2) = 'shell'
overshoot_bdy_loc(2) = 'any'
overshoot_f(2) = 0.0174
overshoot_f0(2) = 0.00174
overshoot_scheme(3) = 'exponential' ! carbon
  ↳ burning
overshoot_zone_type(3) = 'any'
overshoot_zone_loc(3) = 'any'
overshoot_bdy_loc(3) = 'bottom'
overshoot_f(3) = 0.005
overshoot_f0(3) = 0.0005
...

```

b.2 Temmink et al. (2023)

```
! inlist_common
&controls
...
overshoot_scheme(1) = 'exponential'
overshoot_zone_type(1) = 'any'
overshoot_zone_loc(1) = 'core'
overshoot_bdy_loc(1) = 'top'
overshoot_f(1) = 0.016d0
overshoot_f0(1) = 0.08d0

overshoot_scheme(2) = 'exponential'
overshoot_zone_type(2) = 'any'
overshoot_zone_loc(2) = 'shell'
overshoot_bdy_loc(2) = 'any'
overshoot_f(2) = 0.0174d0
overshoot_f0(2) = 0.0087d0
...

```

b.3 Temmink et al. (2023) other $f_{\text{ov},0}$

```
! inlist_common
&controls
...
overshoot_scheme(1) = 'exponential'
overshoot_zone_type(1) = 'any'
overshoot_zone_loc(1) = 'core'
overshoot_bdy_loc(1) = 'top'
overshoot_f(1) = 0.016d0
overshoot_f0(1) = 0.008d0

overshoot_scheme(2) = 'exponential'
overshoot_zone_type(2) = 'any'
overshoot_zone_loc(2) = 'shell'
overshoot_bdy_loc(2) = 'any'
overshoot_f(2) = 0.0174d0
overshoot_f0(2) = 0.0087d0
...

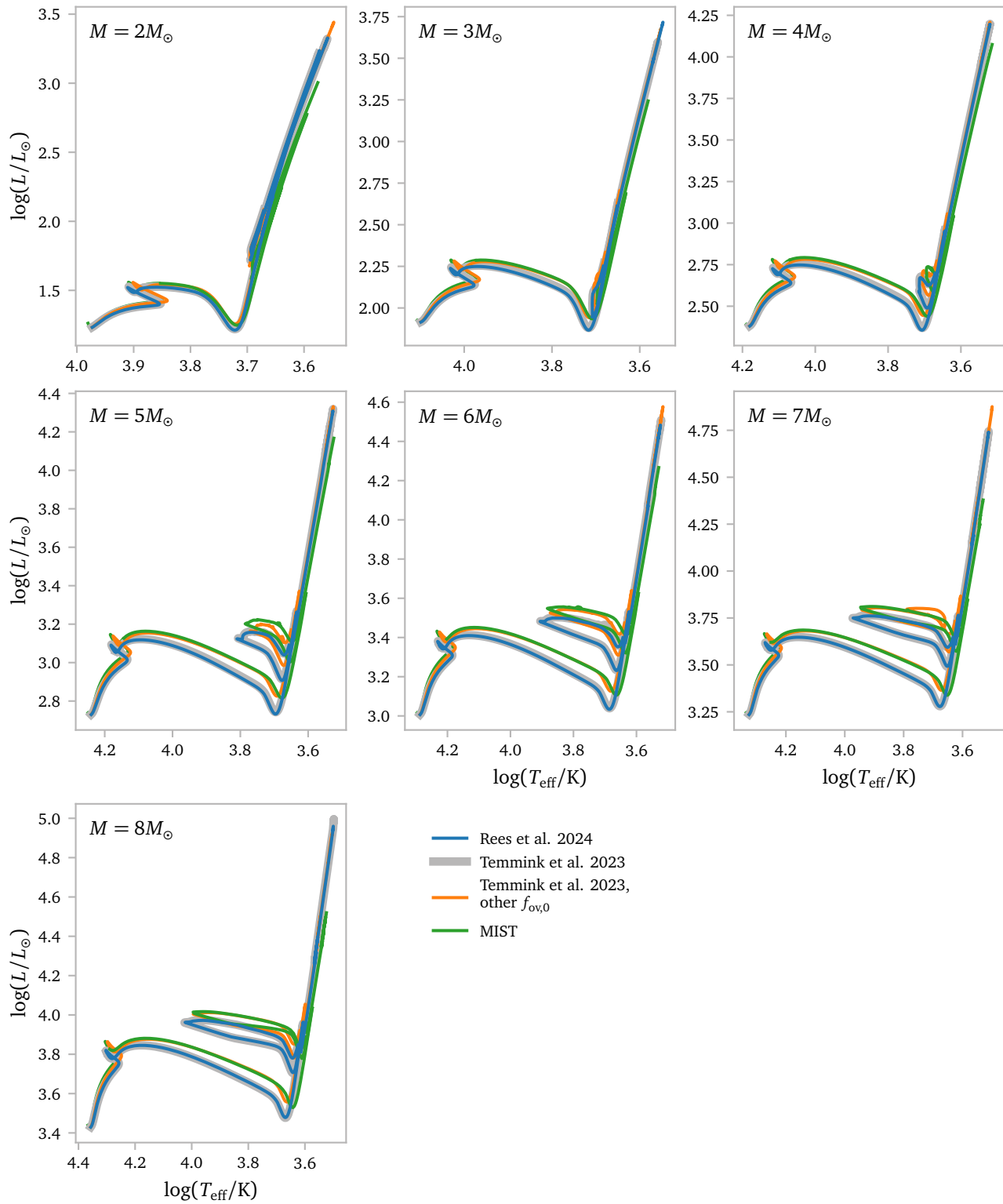
```


b.4 Results

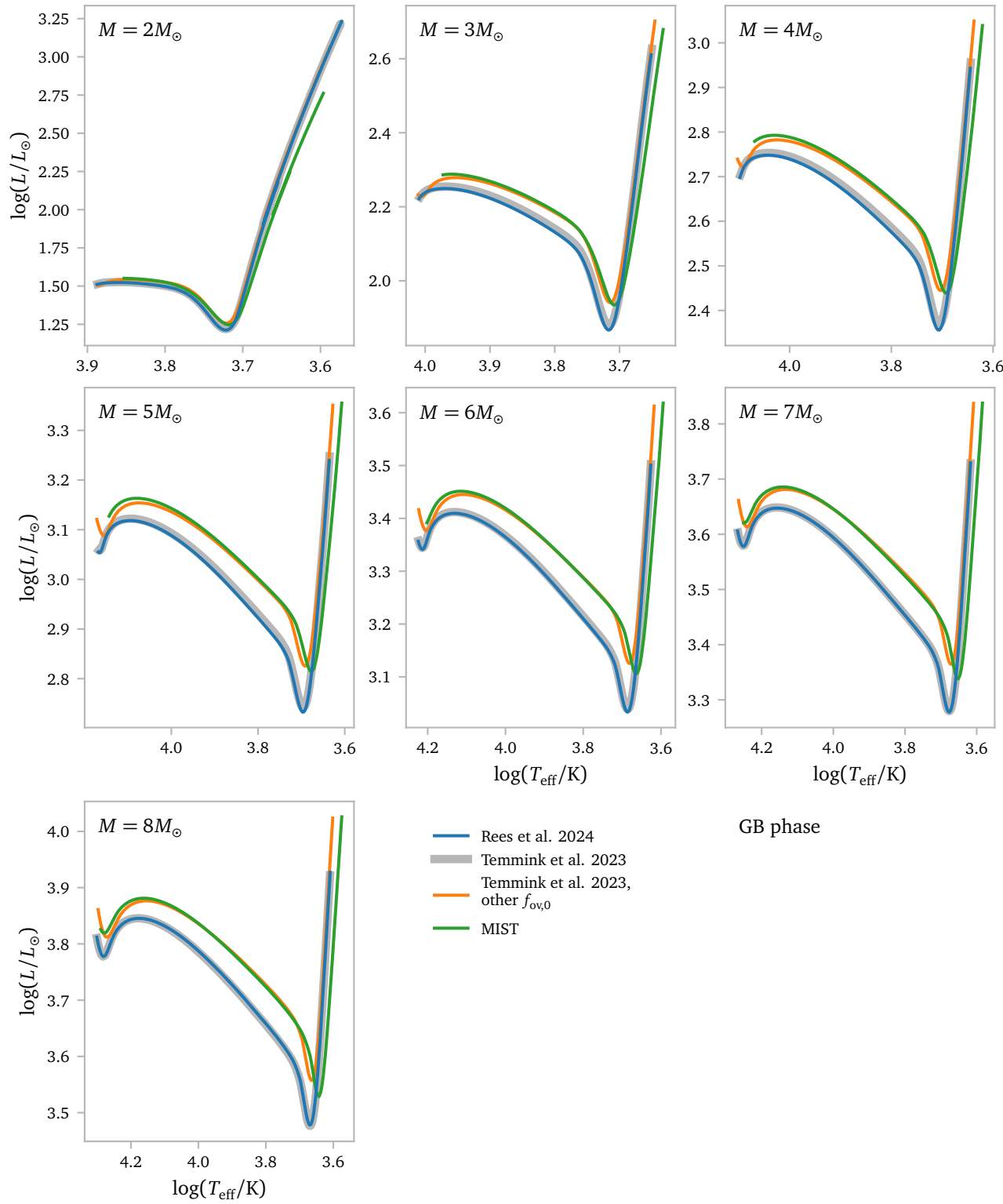
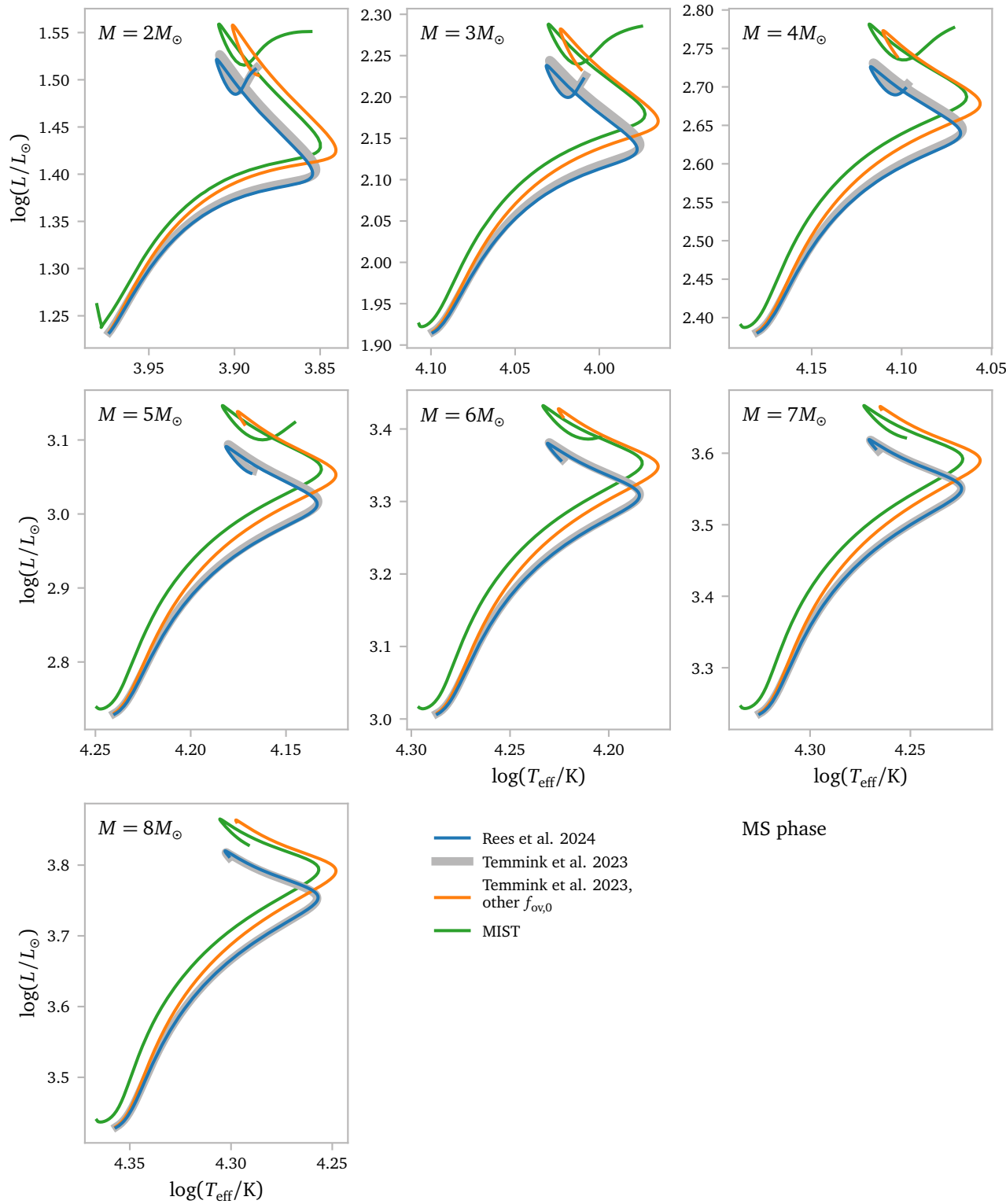
b.4.1 HR-diagrams

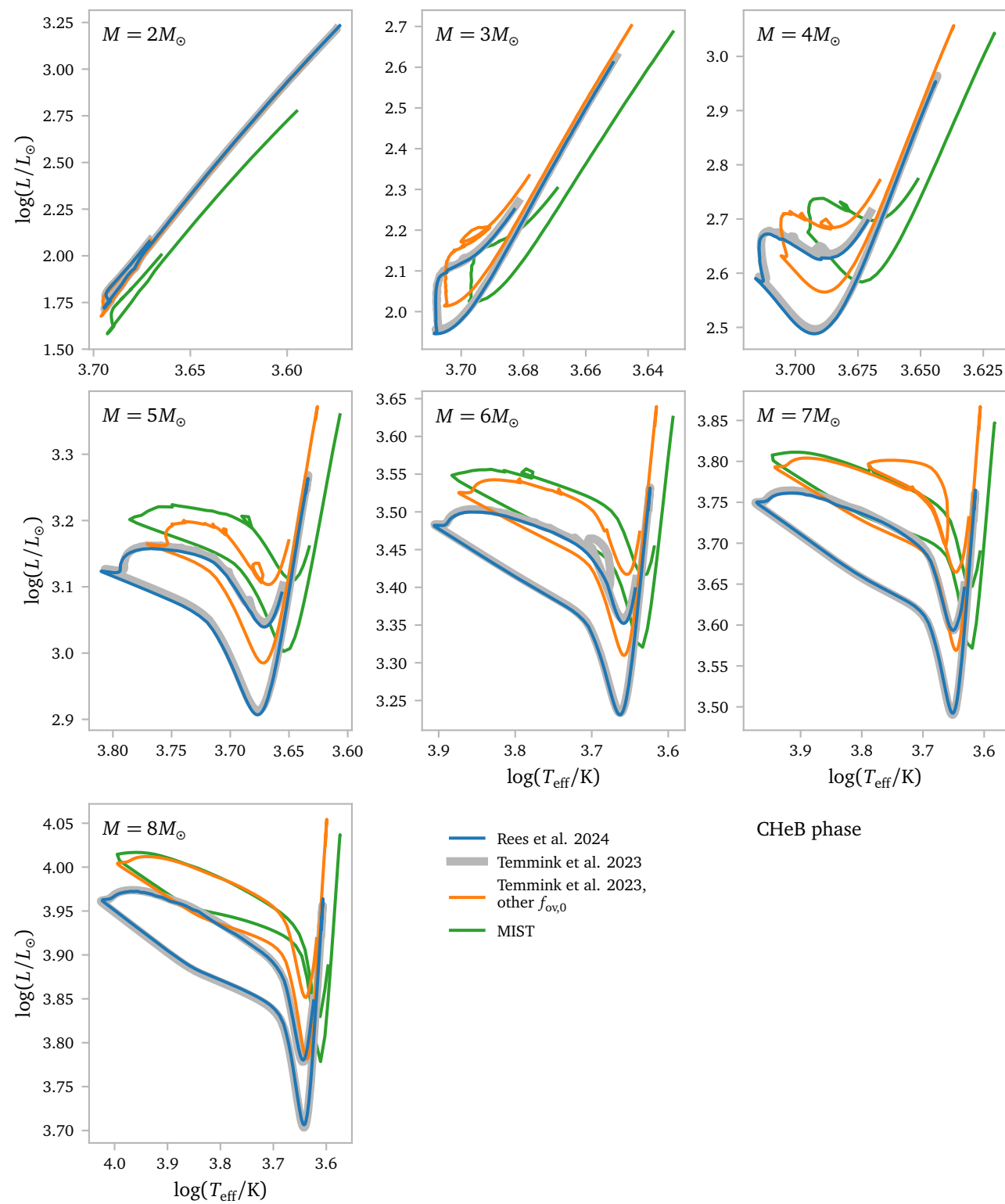
Below, I show Hertzsprung Russel diagrams for the pre-TPAGB phases. For 7 different masses, the overshooting options listed above are shown.

I also compare these tracks with MIST (Choi et al., 2016) EEP tracks.



Now, it is quite difficult to see real differences in these small plots. Therefore, I plot the phases separately as well.





The tracks in the HR-diagrams for stars using the overshooting options from [Rees et al. \(2024\)](#) and [Temmink et al. \(2023\)](#) are very similar. When using the other $f_{\text{ov}0}$, the results are however significantly different and closer to MIST EEPs. In general, these stars have higher luminosities.

For the MIST ([Choi et al., 2016](#)) EEP tracks, I used a set of non-rotating $[\text{Fe}/\text{H}] = 0$ models. This corresponds with an initial $Z = 0.01429$, slightly higher than the $Z = 0.014$ used for my MESA models. The MIST models also use Ferguson low-T opacities as well as a slightly different mixing length parameter $\alpha_{\text{MLT}} = 1.82$ instead of $\alpha_{\text{MLT}} = 1.931$ from [Cinquegrana et al. \(2022\)](#). Maybe these differences could account for the differences visible between the MIST and [Temmink et al. \(2023\)](#) models with alternative $f_{\text{ov}, 0}$.

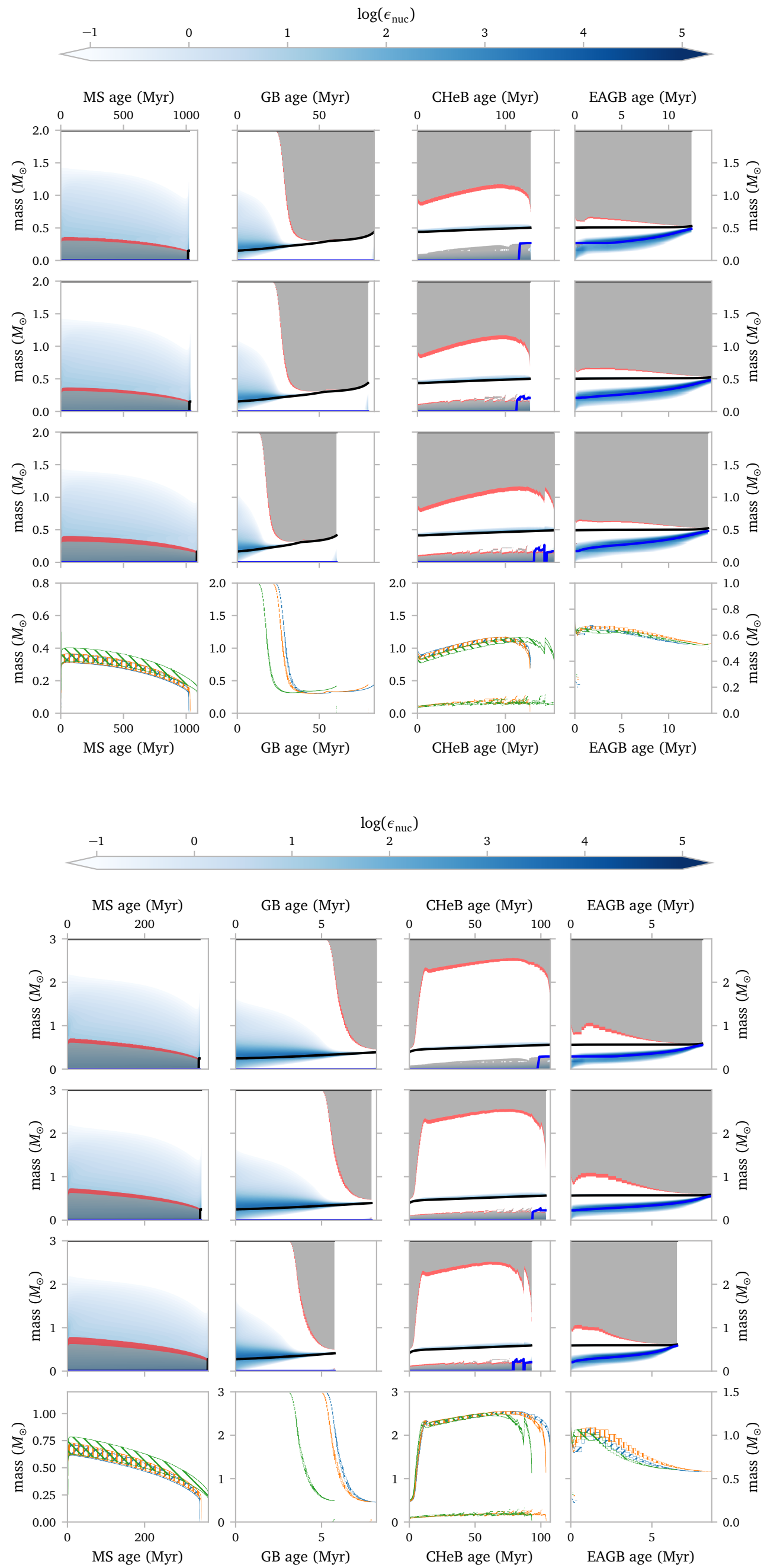
b.4.2 Kippenhahn diagrams

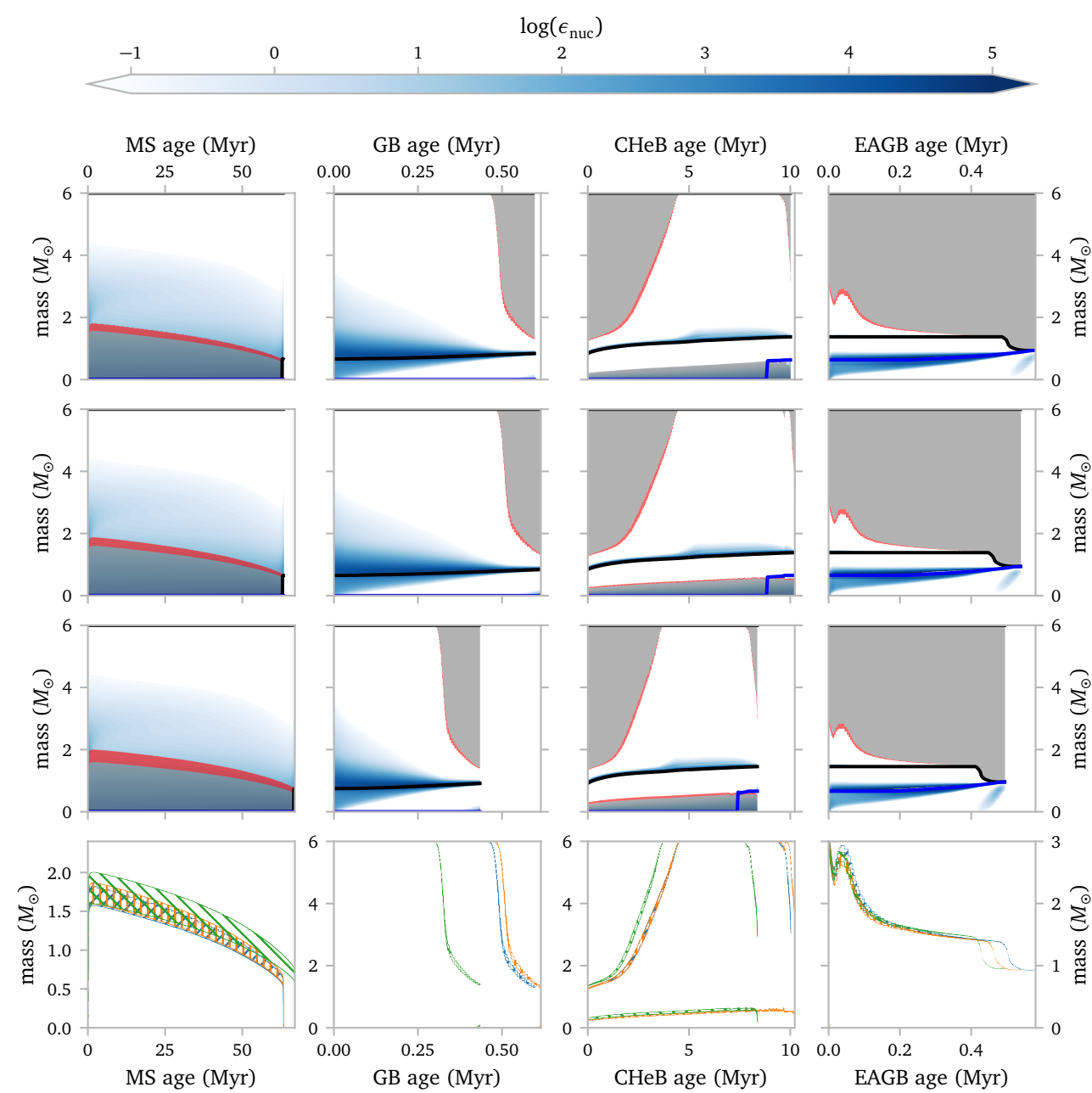
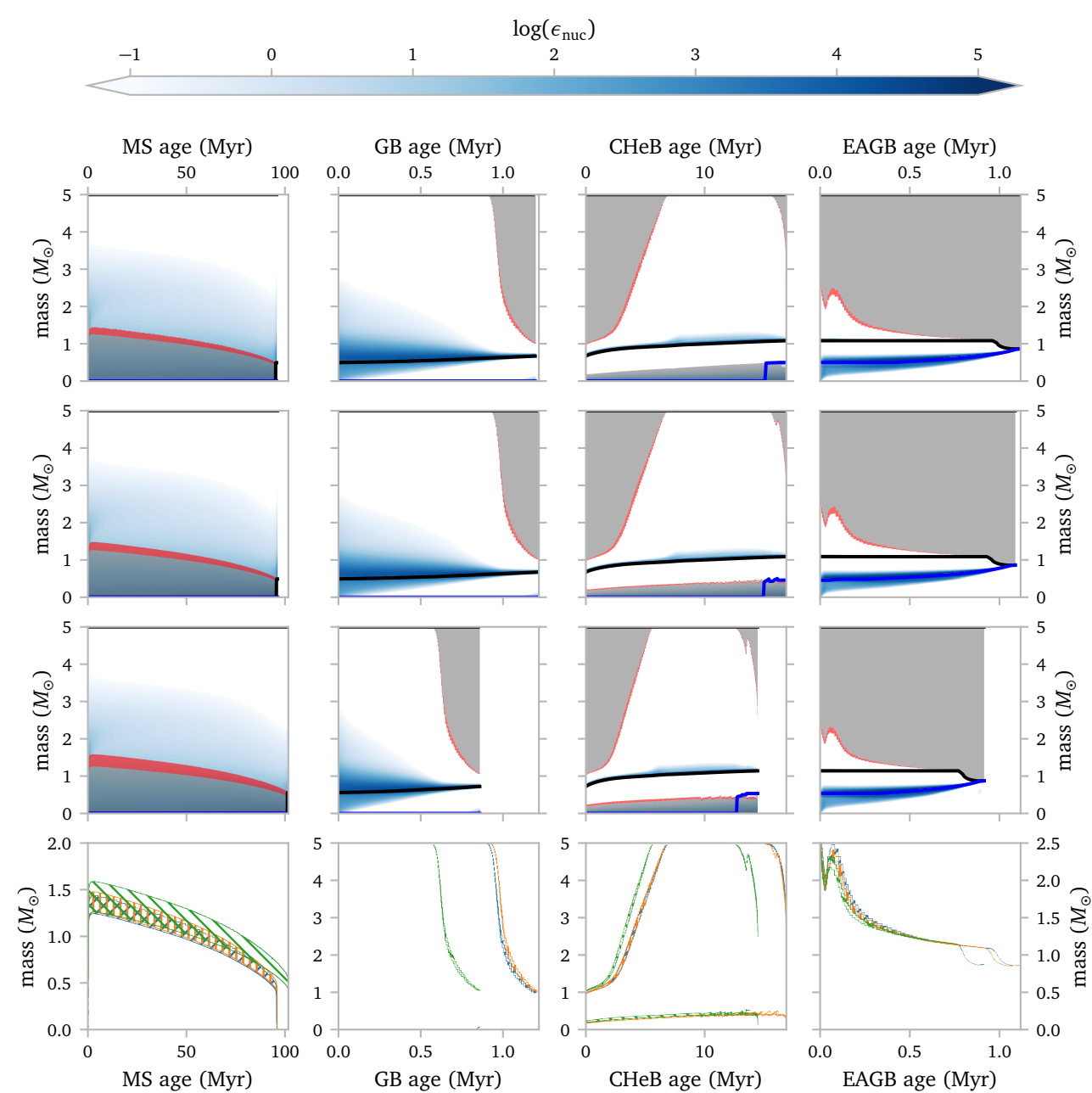
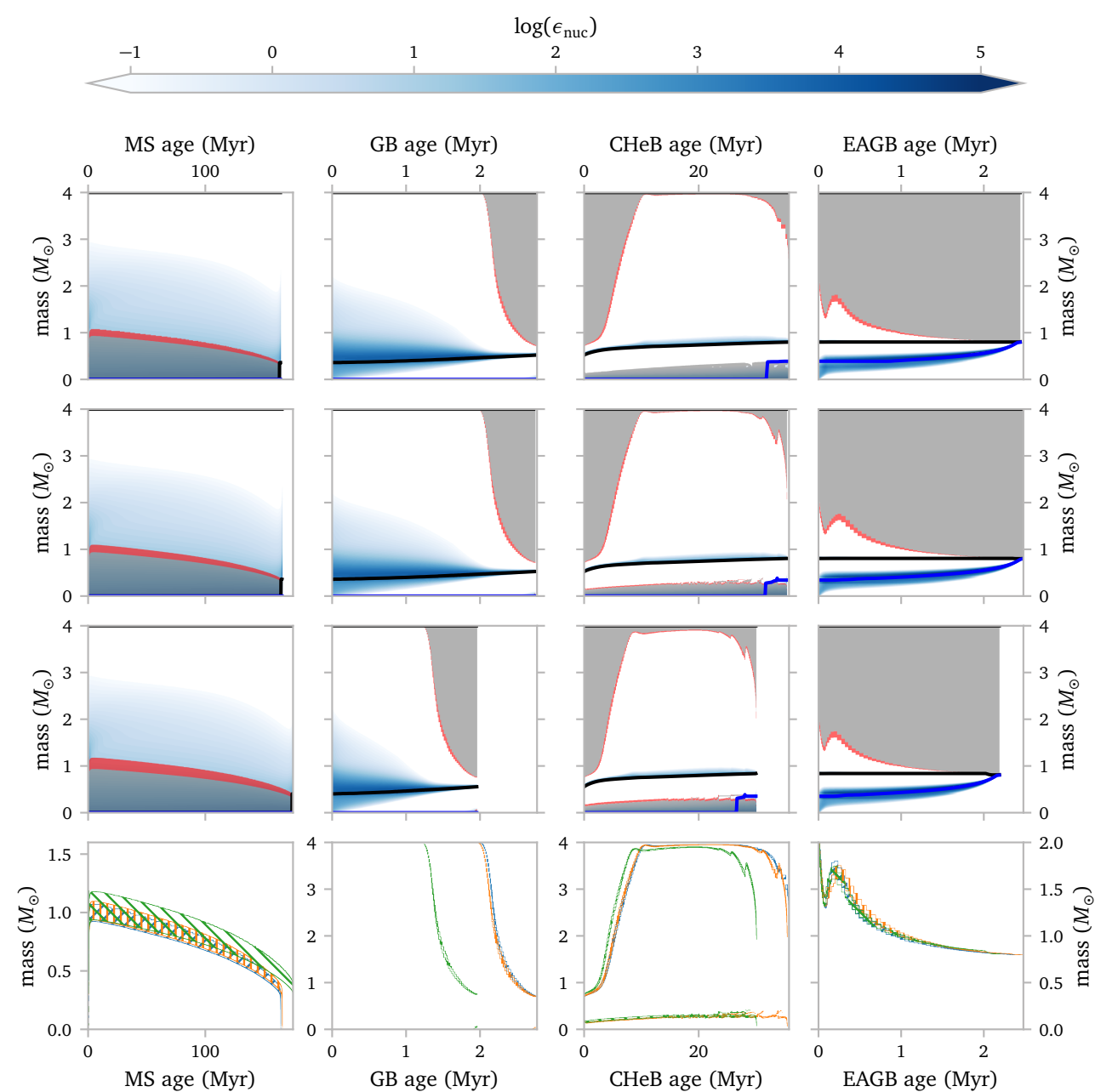
Below, I show Kippenhahn diagrams of the evolutionary stages for all stars. For completeness, I plot all of them, but really, the different masses are not very different².

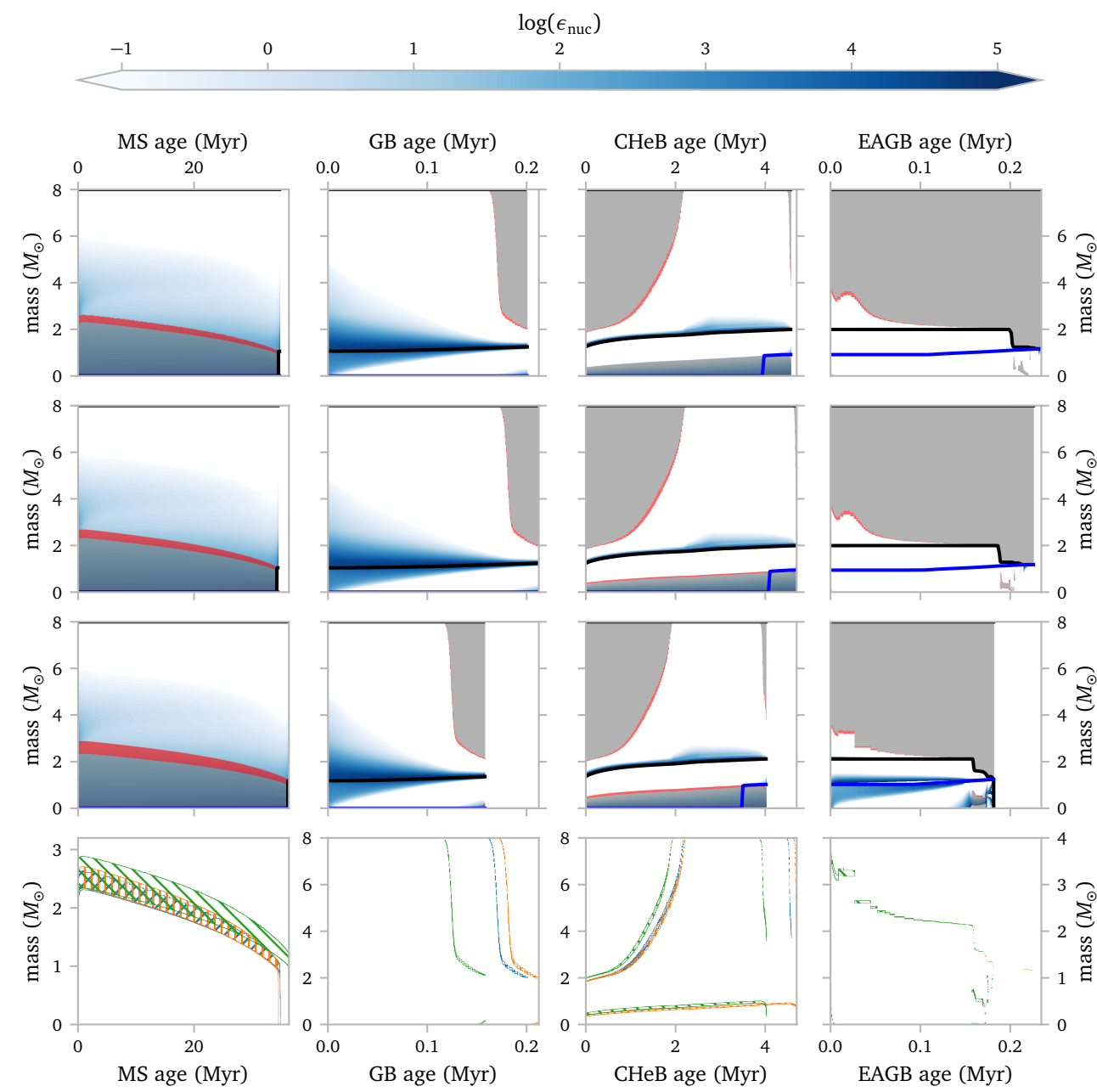
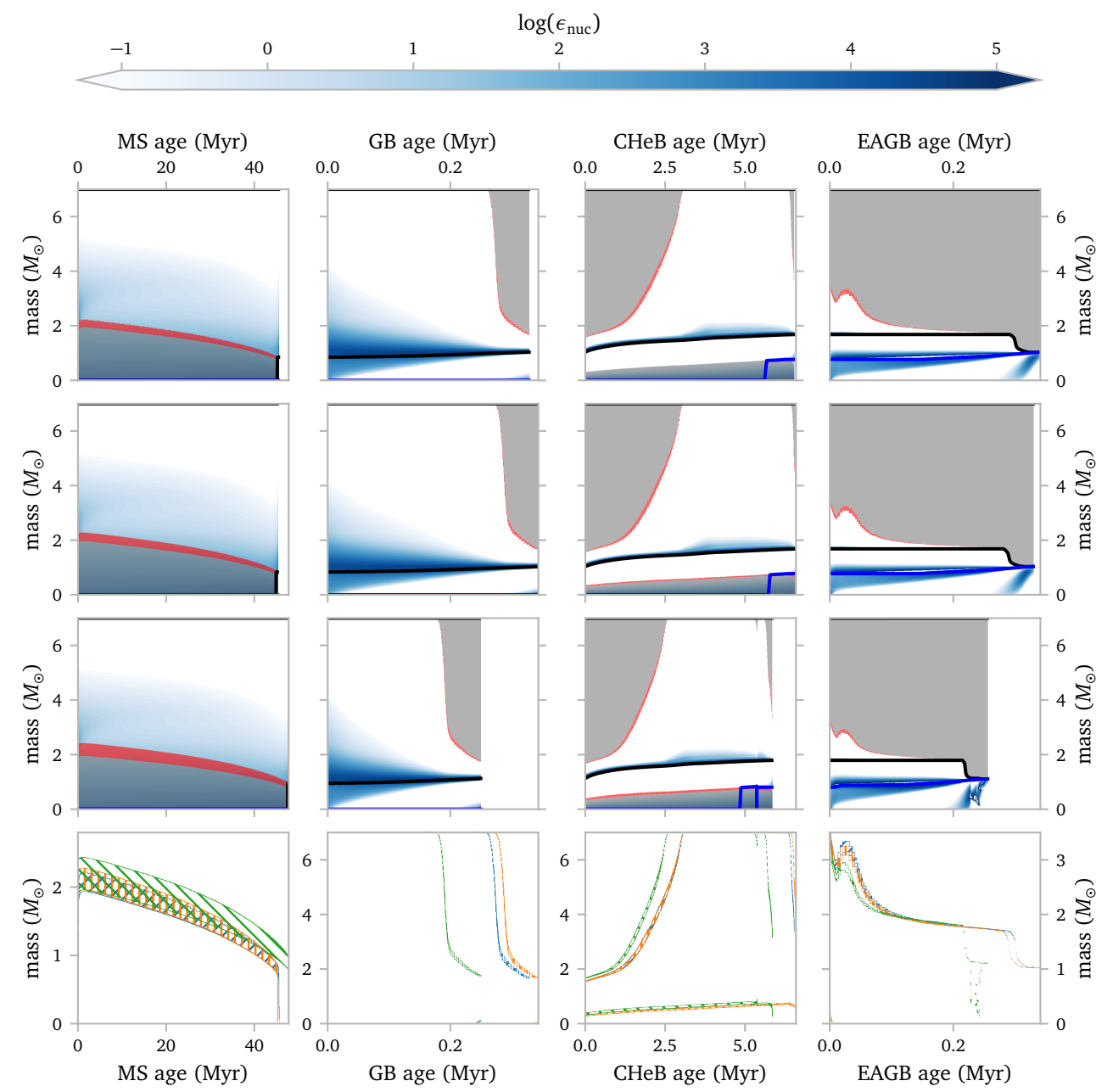
The gray regions are convective regions in the star. The red regions are overshooting regions. The blue "contours" show where nuclear burning is happening. The black line is the He-core mass and the blue line is the CO-core mass.

The last row compares the overshooting regions. The blue regions are the overshooting regions from [Rees et al. \(2024\)](#). The orange regions are the overshooting regions from [Temmink et al. \(2023\)](#) and the green regions are the overshooting regions from [Temmink et al. \(2023\)](#) with a different value of $f_{\text{ov}, 0}$.

² It took me quite some time to figure out that the `mkipp` code that I was using, used MESAs old definitions of mixing regions, while my models used the new definitions. I therefore had to change the `mkipp` code to use the newer definitions.







The core mass at the start of the TPAGB is indistinguishable for the different models. Does this mean it is not very important which prescription is used?

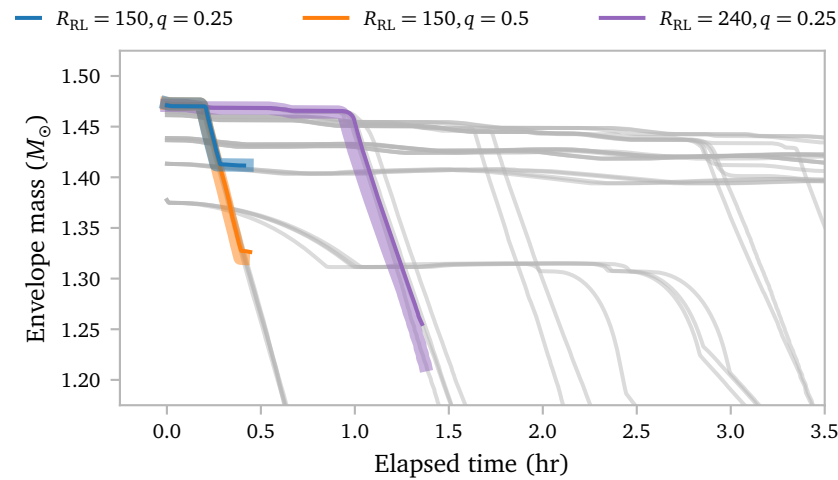
c. Opacity tables

To see if using the [Ferguson et al. \(2005\)](#) opacity table might improve stability during episodes of fast mass transfer during a thermal pulse, I compare binary models with donor TPAGB stars using the [Ferguson et al. \(2005\)](#) opacity tables, and the [Marigo & Aringer \(2009\)](#) opacity tables.

I set up a grid for 24 models for both stars where for every initial Roche lobe radius, I simulate the binary for $q = 0.25, 0.5, 0.75$ and 1. For the radii, I chose $R_{\text{RL}}(R_{\odot}) = 150, 240, 330, 420, 510$ and 600.

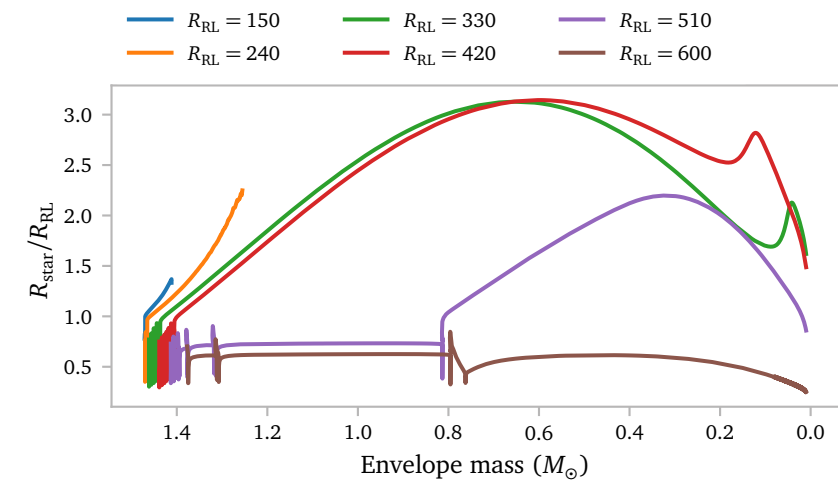
Unfortunately, it quite quickly became clear that both models have trouble with stability at low q and low R_{RL} .

Below, I plot which models failed. The thick lines correspond to $\text{\AA}$ SOPUS models, while the thin lines correspond to Ferguson models.



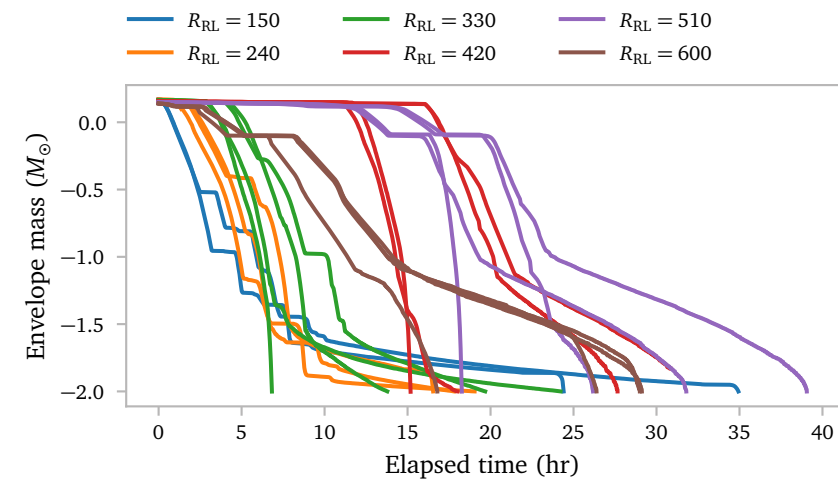
I think it is interesting that the problem only occurs for small radii. Maybe this has to do with the ratio of star radius versus Roche lobe radius.

To test my hypothesis, I plot the ratio of star radius over Roche lobe radius for $q = 0.25$ and different starting radii for the Ferguson models.



There does seem to be a correlation between the initial Roche lobe radius and the ratio $R_{\text{star}}/R_{\text{RL}}$.

I also plot elapsed time for all Ferguson models.

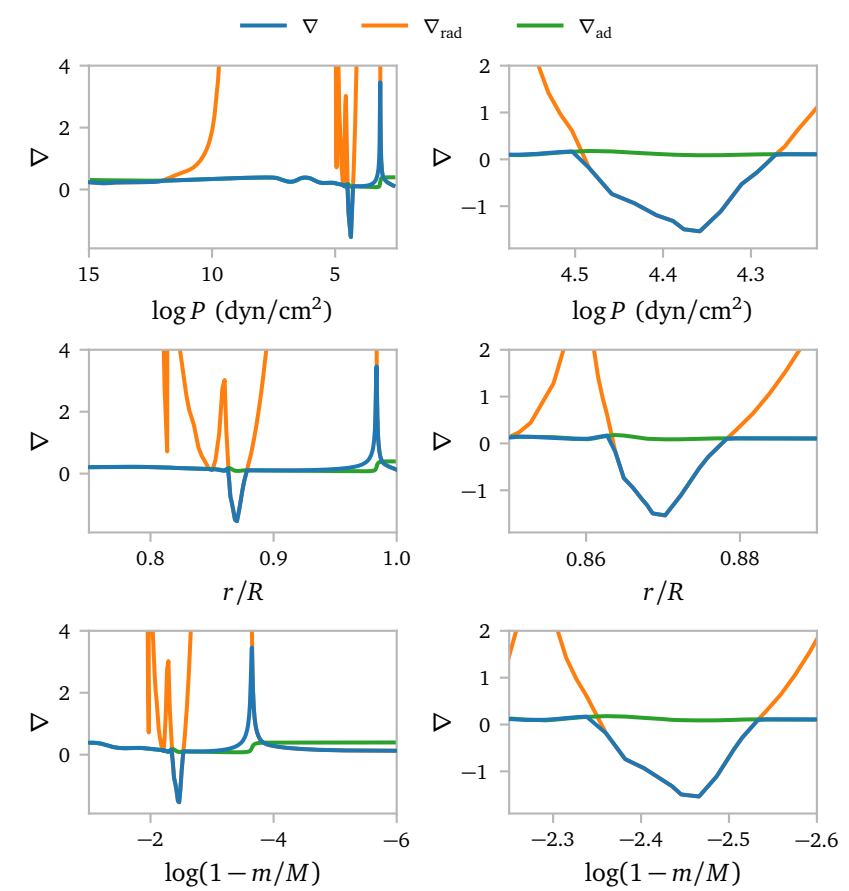


Two things here are interesting:

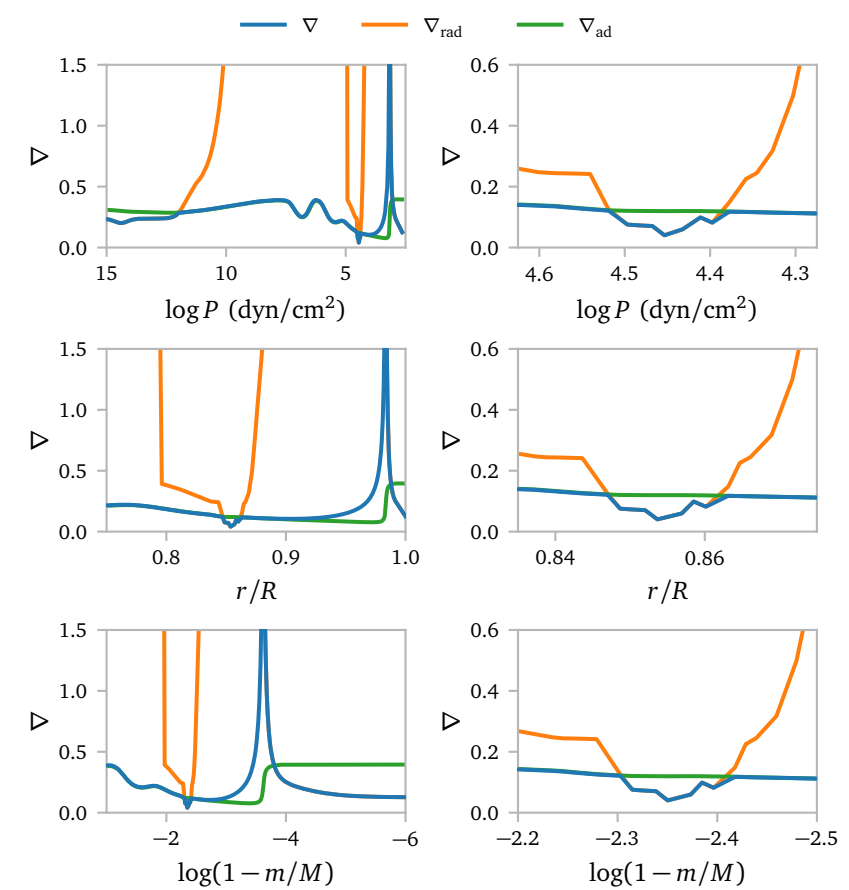
- Generally, models with a larger starting Roche lobe radius take longer to start losing mass.
- The last part of the evolution can take a significant portion of the computing time, even when ending at $0.01M_{\odot}$.

I also check if the temperature gradient problems are present for both failing [Ferguson et al. \(2005\)](#) models, as well as failing [ÆSOPUS \(Marigo & Aringer, 2009\)](#) models. Unfortunately, this is the case, as can be seen below:

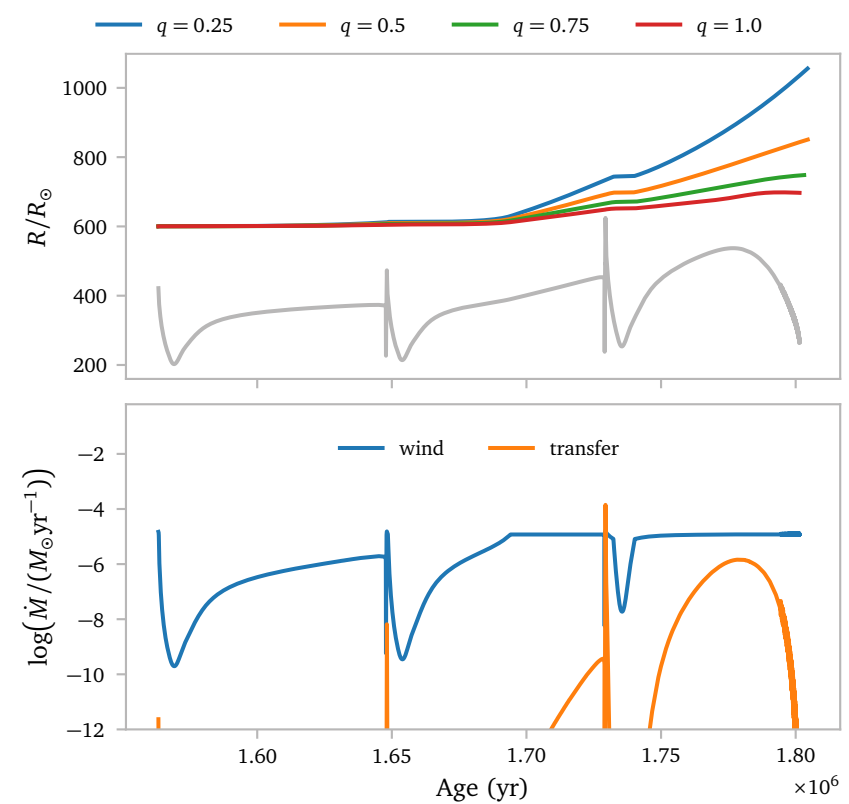
Ferguson $R_{\text{RL}} = 150$, $q = 0.25$:



ÆSOPUS $R_{\text{RL}} = 150$, $q = 0.25$



I also thought the $R_{\text{RL}} = 600R_{\odot}$ model looked interesting in the plots above. Below I plot the evolution of the Roche lobe:

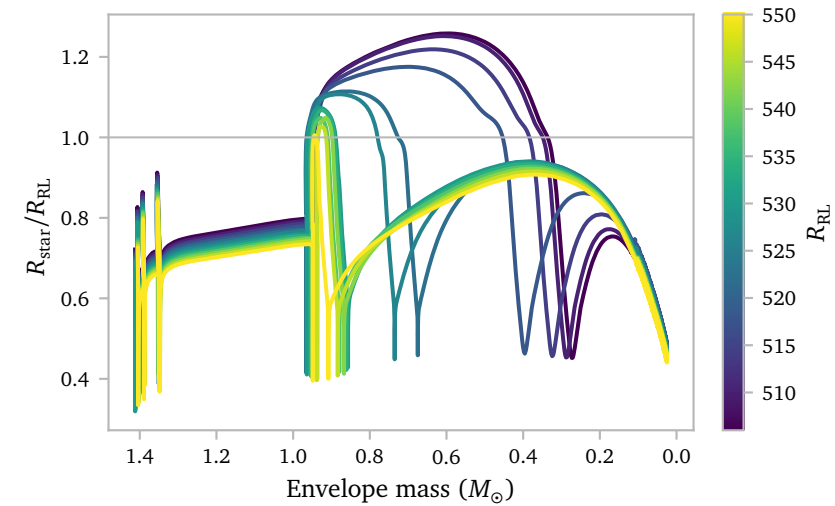


I think this figure quite clearly shows that in parts where wind mass loss is important, the Roche lobe radius will increase significantly as a result. In extreme cases, this can mean that the star just barely avoids filling its Roche lobe.

d. Close R_{RL} Spacing

I also simulated a grid of models with $q = 0.8$ and R_{RL} ranging from $506R_{\odot}$ to $550R_{\odot}$.

These models just finished this morning and I only had time to make a basic plot without looking too much into the models too much.



References

- Choi, J., Dotter, A., Conroy, C., et al. (2016) *Mesa Isochrones and Stellar Tracks (MIST). I. Solar-scaled Models*. [ApJ823](#), 102.
- Cinquegrana, G. C., Joyce, M., & Karakas, A. I. (2022) *Bridging the Gap between Intermediate and Massive Stars. I. Validation of MESA against the State-of-the-Art Monash Stellar Evolution Program for a $2M_{\odot}$ AGB Star*. [ApJ939](#), 50.
- Ferguson, J. W., Alexander, D. R., Allard, F., et al. (2005) *Low-Temperature Opacities*. [ApJ623](#), 585.
- Marigo, P. & Aringer, B. (2009) *Low-temperature gas opacity. ÆSOPUS: a versatile and quick computational tool*. [A&A508](#), 1539.
- Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up*. [MNRAS527](#), 9643.
- Temmink, K. D., Pols, O. R., Justham, S., Istrate, A. G., & Toonen, S. (2023) *Coping with loss. Stability of mass transfer from post-main-sequence donor stars*. [A&A669](#), A45.